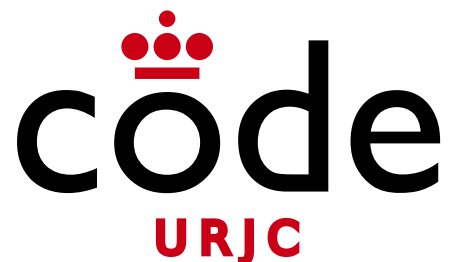


Fundamentos de la Web

Bloque IV: Tecnologías de interactividad
en el cliente web

Tema 4.1: HTML interactivo con JavaScript



©2025

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

JavaScript en el navegador

- Las **páginas web** pueden incorporar **interactividad** con el lenguaje **JavaScript**
- Con **JavaScript** se puede **modificar la página** (manipulando modelo de objetos del documento **DOM**)
- También se pueden hacer **peticiones al servidor web** en segundo plano y **actualizar el contenido de la web** con los resultados (AJAX)

JavaScript en el navegador

- JavaScript se creó en **1995** por **Brendan Eich** para añadir interactividad al navegador web **Netscape**
- Se llamó **Mocha**, luego **LiveScript**, luego **JavaScript** y el estándar es **ECMAScript**
- En **2009** se creó **Node.js** para su ejecución fuera del navegador
- Con la versión **ES6/ES2015** JavaScript comenzó su modernización y se expandió su uso

JavaScript en el navegador

- Compatibilidad de los navegadores con las últimas versiones de JavaScript
- Actualmente los **navegadores modernos** (Chrome, Derivados de Chrome, Firefox y Safari) soportan las características más modernas de JavaScript
- Web para revisar el soporte JavaScript de los navegadores y *runtimes*

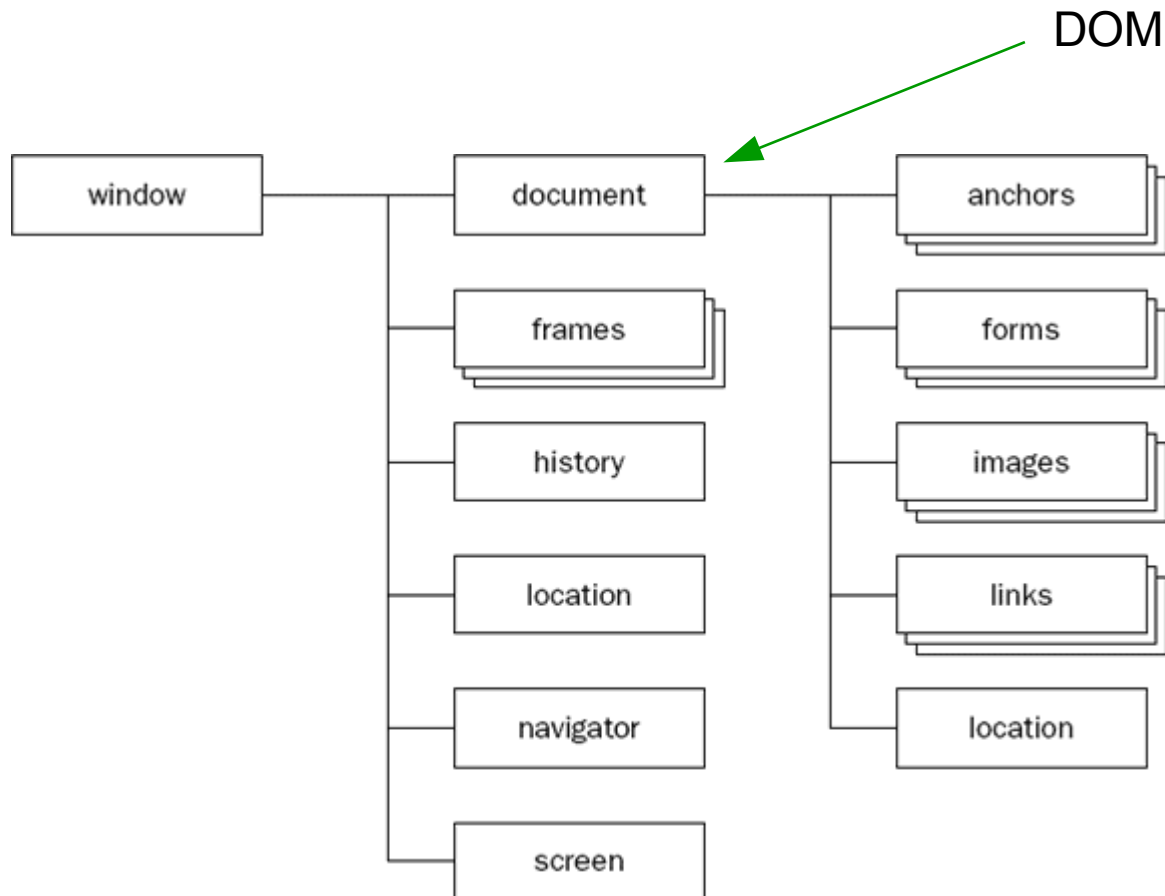
<https://compat-table.github.io/compat-table/es2016plus/>

Librerías JavaScript

- **DOM (*Document Object Model*)**
 - Biblioteca usada para manipular el documento HTML cargado en el navegador
 - Permite insertar y eliminar elementos, la gestión de eventos, cambiar propiedades..
- **BOM (*Browser Object Model*)**
 - Biblioteca usada para usar otros elementos del browser: historial, peticiones de red AJAX, etc...
 - El BOM tiene como propiedad al DOM

Librerías JavaScript

- BOM (*Browser Object Model*)



Librerías JavaScript

- Para programar se pueden usar diferentes librerías
 - **Librería del lenguaje:** String, Array, Map, Date, Math...
 - **Librería del navegador:** DOM, BOM, fetch...
 - **Librerías de terceros (comunidad o compañías):**
 - Permiten reutilizar código desarrollado por otros
 - Se declaran en el HTML o se gestionan con Node.js y un empaquetador (*bundler*)

Librerías JavaScript

- Librerías muy usadas cuando **JavaScript era muy limitado** y los navegadores **no soportaban** correctamente el **estándar** (actualmente en **desuso**)



UNDERSCORE.JS

- **jQuery**: Ofrece una forma alternativa de acceder al API DOM que aporta facilidad de uso y compatibilidad entre navegadores. También permite realizar peticiones ajax.
- **underscore.js**: Librería para trabajar con estructuras de datos con un enfoque funcional. También permite gestionar plantillas (*templates*) para generar HTML partiendo de datos

Frameworks de alto nivel

- Existen **frameworks de alto nivel** que permiten desarrollar **aplicaciones SPA**



Referencias

- **Documentación**

- <https://developer.mozilla.org/es/docs/Web/JavaScript>
 -

- **Tutoriales interactivos**

- <https://www.codecademy.com/>
 - <https://www.pluralsight.com/courses/javascript-gettin>

Integración con HTML

Ejemplo1

- El código **JavaScript** se incluye en etiquetas `<script>` en el `<body>` o `<head>`
- Se puede incluir en el fichero HTML o en ficheros .js
- Se puede añadir en varios lugares (pero no es necesario)

index.html

```
<html>
  <head>
    <script>console.log("Head1");</script>
    <script src="app1.js"></script>
  </head>
  <body>
    <script>console.log("Body1");</script>
    <script src="app2.js"></script>
  </body>
</html>
```

app1.js

```
console.log("Head2");
```

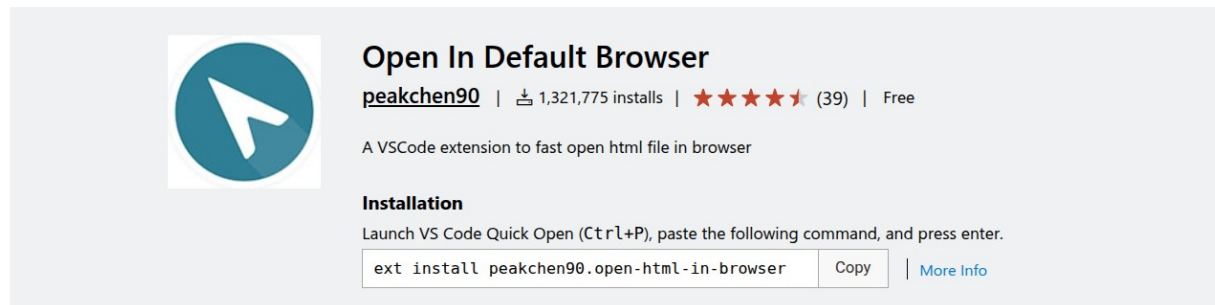
app2.js

```
console.log("Body2");
```

Integración con HTML

Ejemplo1

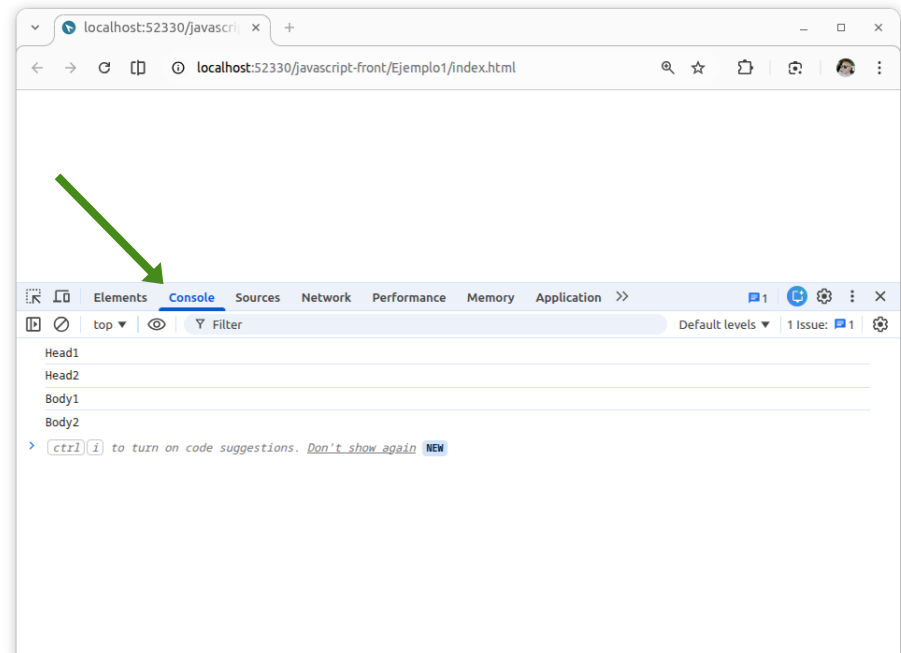
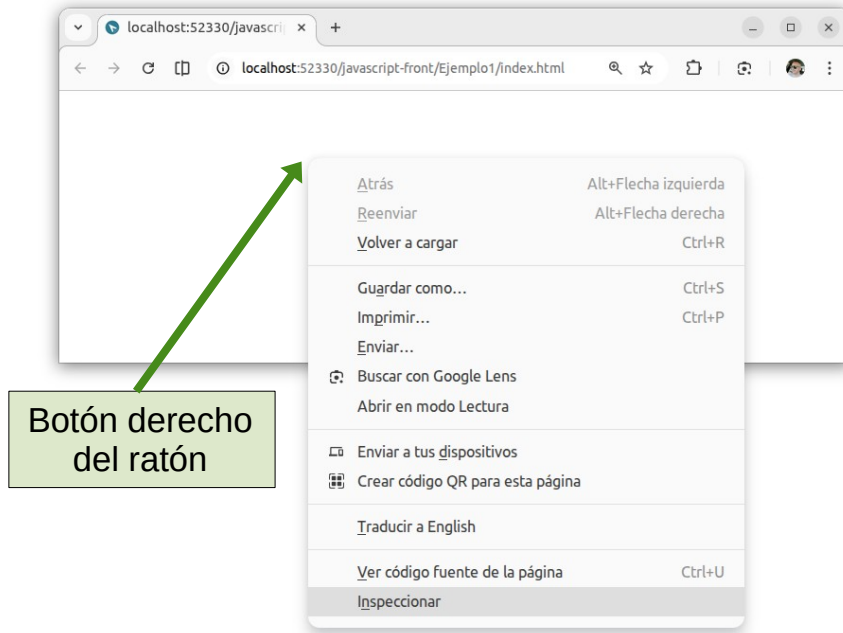
- Para poder utilizar todas las **funcionalidades de JavaScript** es mejor que el fichero HTML sea cargado desde un **servidor web** en vez del **sistema de ficheros** (http:// vs file://)
- Se recomienda usar una extensión de VSCode para abrir el fichero



<https://marketplace.visualstudio.com/items?itemName=peakchen90.open-html-in-browser>

Integración con HTML

- Mostrar información desde JavaScript
- Mostrar la consola de Google Chrome



HTML Interactivo

- JavaScript se diseñó para dotar de **interactividad a las páginas HTML** cargadas en el navegador
- Se puede **modificar la página HTML** con JavaScript
 - **Cambio del contenido:** Texto, imágenes, ...
 - **Cambio del estilo:** Cambiando las clases de los elementos, cambiando estilos específicos
- Se puede **ejecutar código** JavaScript cuando el usuario interactúa con la página (**click, hover, etc...**)

Document Object Model (DOM)

- Desde JavaScript se puede acceder al documento y al navegador
 - El **Document Object Model (DOM)** es una librería para manipular documentos HTML
 - El **Browser Object Model (BOM)** permite manipular otros elementos del **navegador**
- <https://developer.mozilla.org/en-US/docs/Web/API/Window>
- <https://developer.mozilla.org/en-US/docs/Web/API/Navigator>

Seleccionar una parte del documento

- Existe una variable global **document** que apunta al documento DOM cargado en el navegador

<https://developer.mozilla.org/en-US/docs/Web/API/Document>

- Se puede obtener una referencia a uno o varios Nodos del árbol DOM usando

```
let image1 = document.getElementById('image1');  
let citas = document.getElementsByClassName('cita');  
let img2 = document.getElementsByName('image2');  
let links = document.getElementsByTagName('a');
```

<https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById>

Modificar el documento con HTML

- Para cambiar el contenido del documento cargado en el browser se obtiene una referencia al nodo y se cambia su contenido HTML con la propiedad **innerHTML**

```
let element = document.getElementById('txt');
element.innerHTML = '<p>Nuevo texto</p>';
```

<https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML>

Modificar el documento con HTML

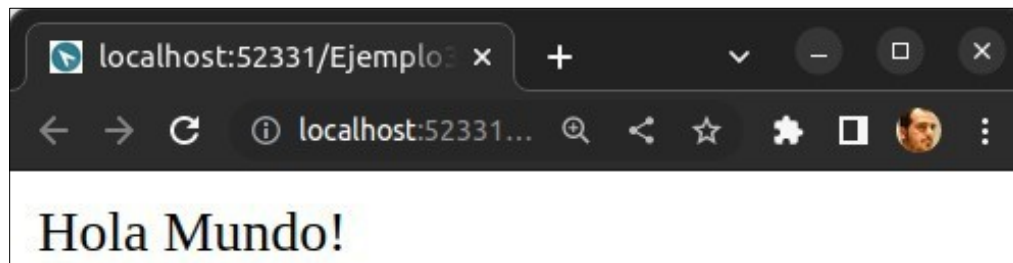
app.js

```
let content = document.getElementById('content');
content.innerHTML = '<p>Hola Mundo!</p>'
```

Ejemplo2

index.html

```
<html>
  <body>
    <div id='content'></div>
    <script src='app.js'></script>
  </body>
</html>
```



Gestión de Eventos

- Se pueden ejecutar **funciones** cuando se interactúa con elementos del documento
- Ejemplo: Mostrar una alerta al pulsar un botón

Ejemplo3

```
function functionAlert() {
    alert('El botón ha sido pulsado');
}
```

```
<html>
  <body>
    <script src='app.js'></script>
    <button onclick="functionAlert();">Botón</button>
  </body>
</html>
```

Gestión de Eventos

Ejemplo4

- Ejemplo: Añadir texto a la página al pulsar un botón

app.js

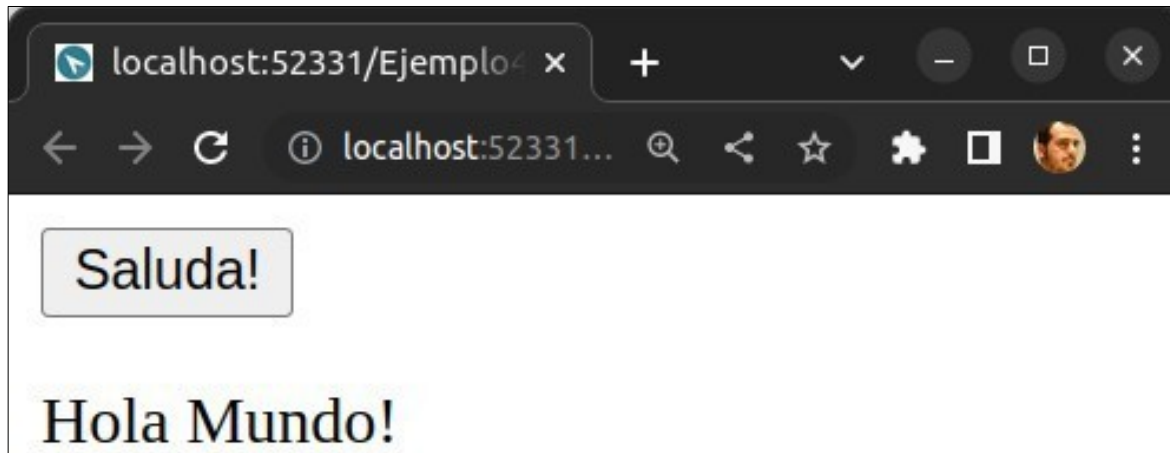
```
function greetings() {
  let content = document.getElementById('content');
  content.innerHTML = '<p>Hola Mundo!</p>';
}
```

index.html

```
<html>
  <body>
    <button onclick="greetings()">Saluda!</button>
    <div id='content'></div>
    <script src='app.js'></script>
  </body>
</html>
```

Gestión de Eventos

Ejemplo4



Gestión de Eventos

- Documentación sobre Eventos en JavaScript y DOM

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events

- Listado de eventos que se pueden producir en un navegador web

<https://developer.mozilla.org/en-US/docs/Web/Events>

- Evento click de un elemento HTML

https://developer.mozilla.org/en-US/docs/Web/API/Element/click_event

Ejercicio 1

- Implementa una web que muestre los títulos de unos libros
 - Cien años de soledad
 - El señor de los anillos
 - 1984
 - Un mundo feliz
- Los títulos de los libros estarán almacenados en un array de JavaScript

Gestión de Eventos

- En vez de configurar el evento en el HTML, también se puede configurar en el código JavaScript con el método **addEventListener**

Ejemplo5

index.html

```
<html>
  <body>
    <button>Saluda!</button>
    <div id='content'></div>
    <script src='app.js'></script>
  </body>
</html>
```

Gestión de Eventos

- En vez de configurar el evento en el HTML, también se puede configurar en el código JavaScript con el método **addEventListener**

Ejemplo5

app.js

```
function greetings() {
    let content = document.getElementById('content');
    content.innerHTML = '<p>Hola Mundo!</p>';
}

let buttons = document.getElementsByTagName('button');

buttons[0].addEventListener('click', greetings);
```

Gestión de Eventos

- La función que se ejecuta cuando se produce un evento (*handler*) puede recibir como parámetro información del evento

Ejemplo6

app.js

```
let buttons = document.getElementsByTagName('button');

function greet(event) {
  console.log('Evento:', event);
  console.log('Elemento origen: ', event.target);
}
```

https://developer.mozilla.org/en-US/docs/Web/Events/Event_handlers

Carga del documento DOM

- Si el script se configura al **final del html**, se ejecuta cuando el documento ya está **cargado** en memoria
- Si el script se configura en el **<head>**, se ejecuta **antes** de que el documento haya sido cargado

```
<html>
  <body>
    ...
    <script src='app.js'></script>
  </body>
</html>
```

```
<html>
  <head>
    <script src='app.js'></script>
  </head>
  <body>
    ...
  </body>
</html>
```

Carga del documento DOM

- Si ponemos el script en el **<head>** hay que esperar a que se haya cargado la página para insertar el contenido desde JavaScript

Ejemplo7

index.html

```
<html>
  <head>
    <script src='app.js'></script>
  </head>
  <body>
    <div id='content'></div>
  </body>
</html>
```

Carga del documento DOM

- El documento tiene el evento **DOMContentLoaded** que se ejecuta cuando el documento se ha cargado
- Si no se espera a la carga del DOM, al buscar el elemento "content" no se encontrará

app.js

Ejemplo7

```
document.addEventListener("DOMContentLoaded", function(){
    let content = document.getElementById('content');
    content.innerHTML = '<p>Hola Mundo!</p>';
});
```

Modificar los estilos

- Se puede cambiar cualquier estilo de un elemento con su propiedad **style**

```
let element = document.getElementById('img1');
element.style.borderWidth = width + 'px';
```

<https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/style>

- Ocultar y mostrar un elemento con estilos

```
let element = document.getElementById('text');
element.style.display = 'none';
...
element.style.display = 'block';
```

Ejercicio 2

- Crea una página con un botón que oculte el texto “Hola Mundo” cuando se pulse
- Si se pulsa de nuevo, se volverá a mostrar el texto “Hola Mundo”

Modificar los estilos

- Una forma común de cambiar los estilos es añadiendo o eliminando clases CSS a un elemento HTML

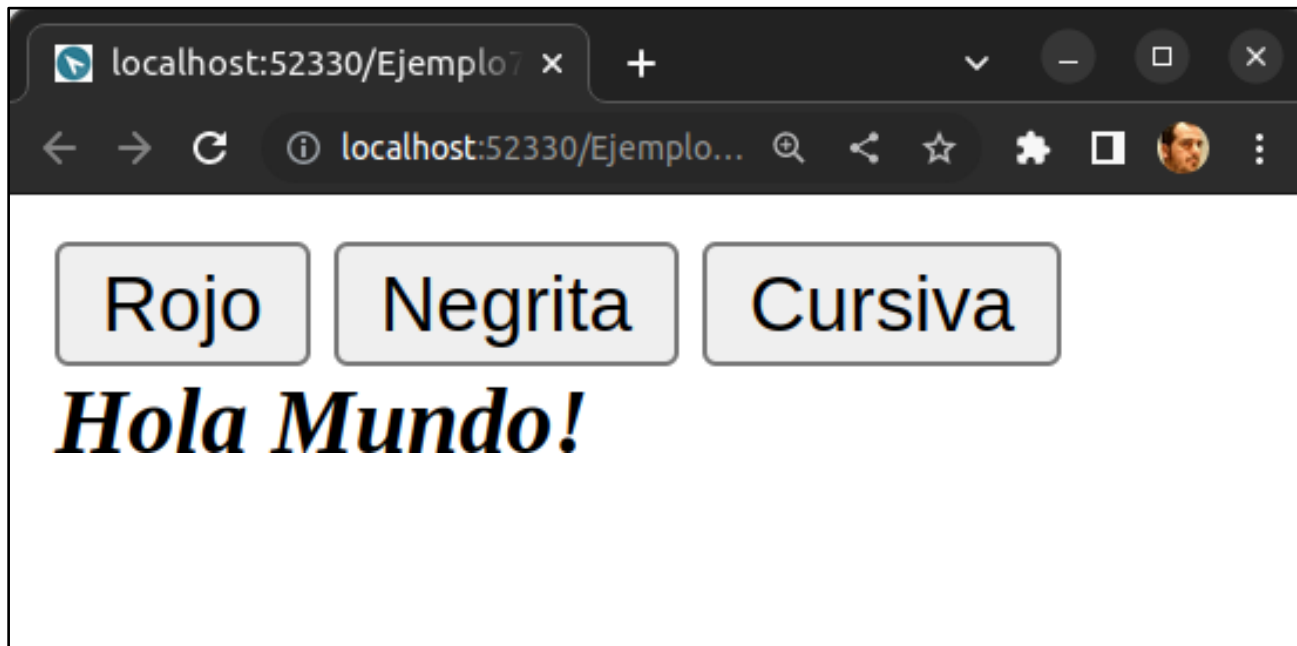
```
let content = document.getElementById('content');
...
content.classList.remove('red');
...
content.classList.add('bold');
...
content.classList.toggle('italic');
```

<https://developer.mozilla.org/en-US/docs/Web/API/Element/classList>

Modificar los estilos

- Ejemplo con botones que añaden o eliminan una clase (*toggle*) de un elemento

Ejemplo8



Modificar los estilos

- Ejemplo con botones que añaden o eliminan una clase (*toggle*) de un elemento

Ejemplo8

```
function red(){
  let content = document.getElementById('content');
  content.classList.toggle('red');
}
```

```
<body>
  <button onclick='red()'>Rojo</button>
  <div id='content'>Hola Mundo!</div>
  ...
</body>
```

```
.red {
  color: #F00;
}
```

Ejercicio 3

- Amplía el Ejercicio 1 para que al lado de cada libro aparezca un botón [Más info]
- Al pulsar el botón aparecerá debajo del título el autor del libro y su año de publicación
- Un elemento se puede ocultar con el estilo “display:none” y mostrarse de nuevo con “display:block”

Ejercicio 3

- Información libros

- Cien años de soledad, de Gabriel García Márquez (1967)
- El señor de los anillos, de J. R. R. Tolkien (1954)
- 1984, de George Orwell (1949)
- Un mundo feliz, de Aldous Huxley (1932)

Generación de HTML

- Existen diversas formas de generar código HTML desde código JavaScript
 - **Cadena de caracteres (*String*)**
 - Creando los nodos del árbol DOM

Generación de HTML

- Cadena de caracteres (*String*)

```
let text = "Hola Mundo!";  
  
let content = document.getElementById("content");  
  
content.innerHTML = "<p>" + text + "</p>";
```

Generación de HTML

- Cadena de caracteres (*String*)
 - Cuando el código HTML es complejo se pueden usar *template literals*
 - Se usan comilla ` (*backtick*)
 - Se usa `\${...}` para las variables
 - Pueden tener varias líneas

```
let html =
  `

<p>${book.title}</p>
    <p> ${book.author} (${book.year})</p>
  </div>`;


```


Generación de HTML

- Existen diversas formas de generar código HTML desde código JavaScript
 - Cadena de caracteres (*String*)
 - **Creando los nodos del árbol DOM**

Generación de HTML

- Creando los nodos del árbol DOM

```
let text = "Hola Mundo!";

let content = document.getElementById("content");

content.innerHTML = "<p>" + text + "</p>";
```



```
let text = "Hola Mundo!";

let content = document.getElementById("content");

let p = document.createElement("p");
p.textContent = text;
content.appendChild(p);
```

Generación de HTML

- Creando los nodos del árbol DOM

Ejemplo

```
let text = "Hola Mundo!";

let content = document.getElementById("content");

let p = document.createElement("p");
p.textContent = text;
content.appendChild(p);
```

<https://developer.mozilla.org/en-US/docs/Web/API/Document/createElement>

<https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent>

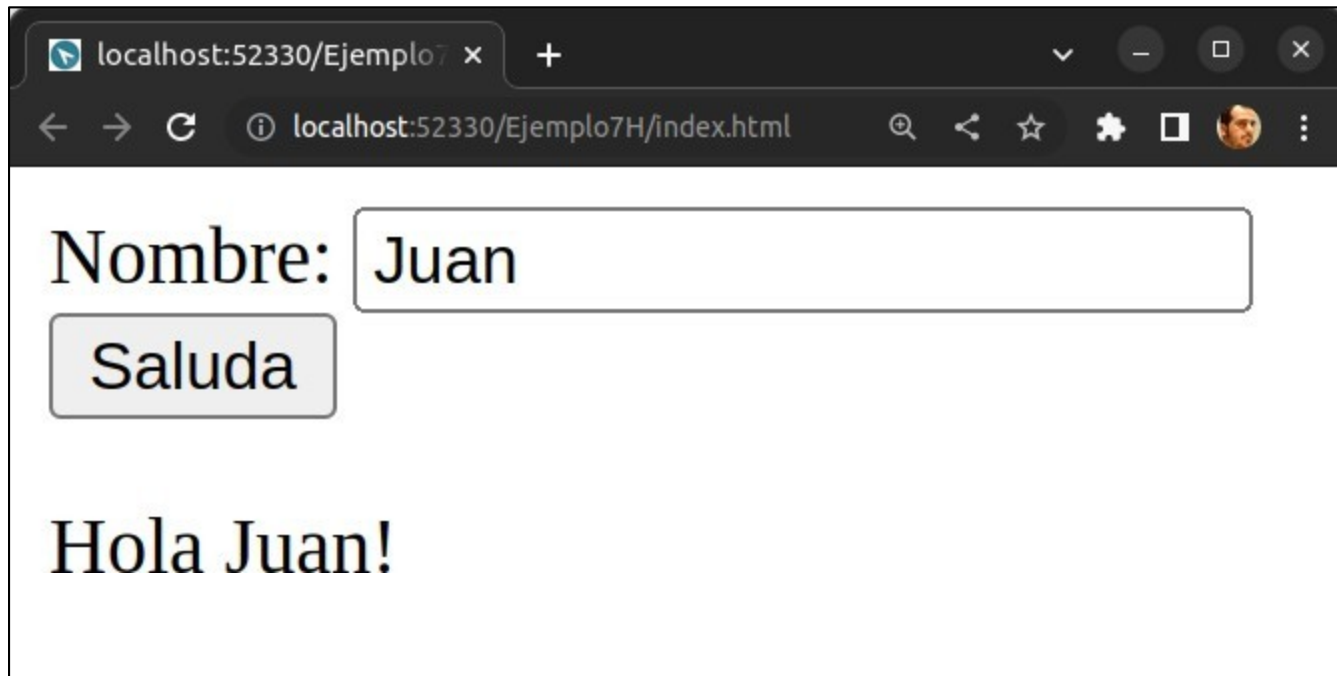
Ejercicio 4

- Actualiza el ejercicio 2 para no generar el HTML como texto y usar la API de nodos del DOM

Formularios

- Se puede obtener información de campos de formularios

Ejemplo10



A screenshot of a web browser window. The address bar shows 'localhost:52330/Ejemplo7H/index.html'. The page content includes a label 'Nombre:' followed by a text input field containing 'Juan'. Below the input field is a button labeled 'Saluda'. Underneath the button, the text 'Hola Juan!' is displayed.

Formularios

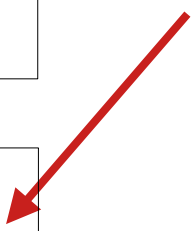
- Se puede obtener información de campos de formularios

Ejemplo10

```
<div>
  Nombre: <input id='name' type='text'></input>
  <button onclick='greetings()'>Saluda</button>
</div>
<div id='content'></div>
```

```
function greetings() {
  let nombre = document.getElementById('name').value;

  let content = document.getElementById('content');
  content.innerHTML += '<p>Hola ' + name + '!</p>';
}
```



Ejercicio 5

- Amplía el ejercicio 4 para que se puedan dar de alta nuevos libros con un formulario

Ejercicio 6

- Crea una página que muestre el título de varios libros
- Al lado de cada título aparecerá el botón “Más info”
- Al pulsar el botón se borrará la lista de libros y aparecerá la información del libro junto con un botón “Volver a la lista”

Ejercicio 6

